

Entrega 2 Proyecto 7

Created by  Andrea

Andrea Carolina Rodriguez Salinas - 202214026

Índice

1. Enlaces Importantes
 - 1.1. Github
 - 1.2. Release
2. Descripción general avance anterior
3. Entrega 2
 - 3.1. Errores Corregidos
 - 3.2. Avance Animación
 - 3.3. Avance Código
 - 3.4. Avance Investigación AR
 - 3.5. Evidencias Visuales
4. Referencias

Enlaces Importantes

Github

https://github.com/acrodriguezs1/Proyecto_Final_CVI

Realease

Descripción general avance anterior

El objetivo central del proyecto se basa en desarrollar un oso de goma interactivo en realidad aumentada. En esta primera entrega, se modeló el oso con textura de goma en Blender y se intentó integrarlo al ejemplo GLTF de Diligent, encontrando problemas como la pérdida de textura y deformaciones del modelo. Para solucionarlo, se comenzó a modificar el código del loader para cargar las texturas directamente, evitando el uso del `load model`. A pesar de múltiples intentos el error persistió. Se definieron las texturas a utilizar, basándose en el tutorial 21, con el objetivo de lograr una apariencia cristalina en el oso sin necesidad de aplicar shaders directamente.

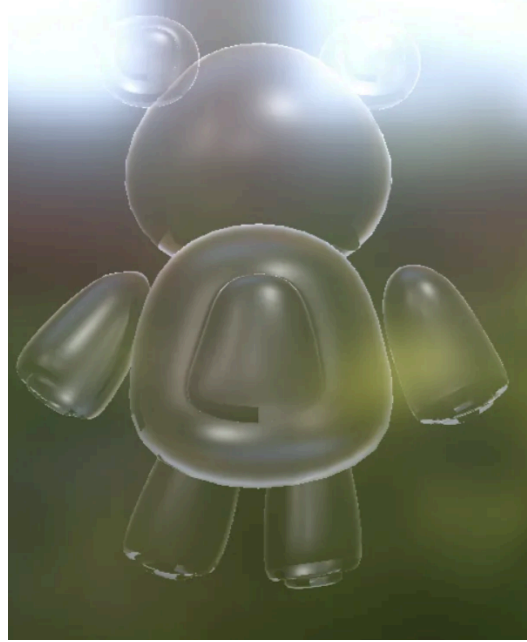
Entrega 2

Errores Corregidos

Para esta entrega se corrigieron los errores evidenciados en la primera, es decir el oso de goma cuando se carga dentro del ejemplo de Diligent GLTF muestra de manera correcta toda la geometría haciendo que el oso este más redondeado además de mostrar la geometría completa es decir la oreja, pierna y brazo faltante ya se evidencian dentro de Diligent.

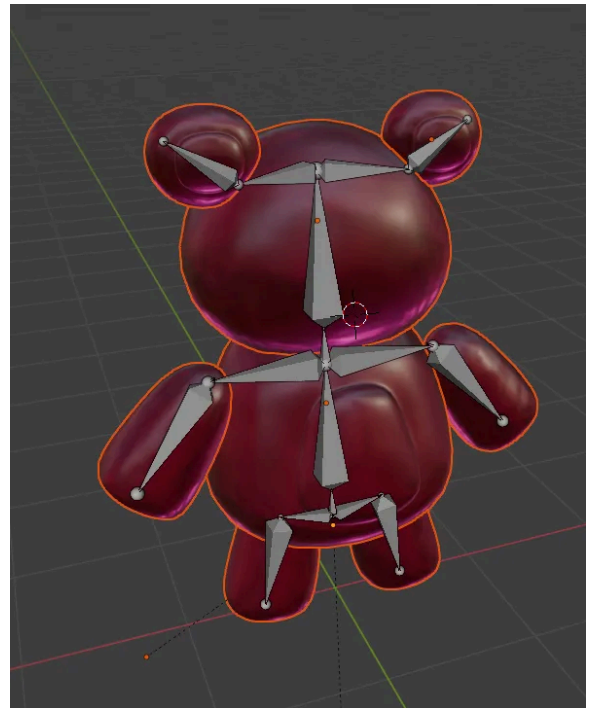
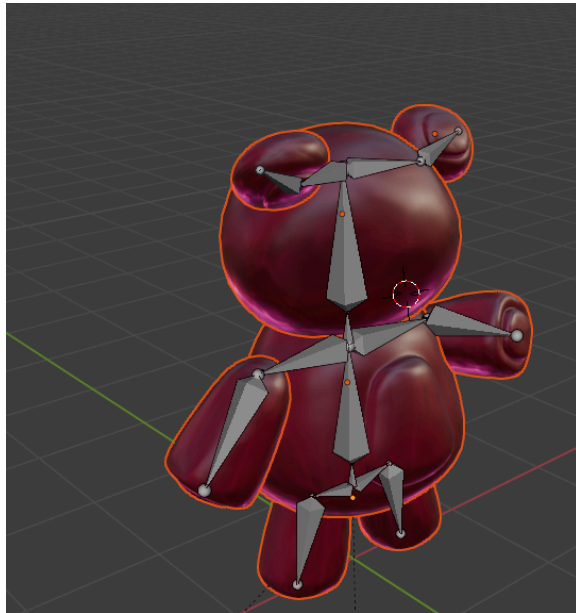
Antes

Después



Avance Animacion

Además, se logro hacer que el oso caminara dentro del programa en Diligent. esto usando rigging para poder realizar el movimiento del oso se realizo una animación en la que el oso avanzaba de posición desde el punto $\{0,0,0\}$ (x,y,z) caminando la posición del oso en el eje y manteniendo su posición en x y z esto genera el movimiento de las articulaciones simulando el movimiento deseado el cual se reduce a una caminata simple moviendo brazos hombros cadera y piernas. Para agregarle un poco más de complejidad se le agrego movimiento a las orejas para simular el movimiento de manera correcta



Avance Codigo

Posteriormente se realizo la animación igualmente en la que se podía ver caminando al Oso, esta al hacer esto se exportaron las texturas y la geometría correctamente nuevamente para ver al oso caminando de manera correcta dentro de Diligent.

En esta parte del código se cambiaron los datos para solamente ingresar los necesarios para generar únicamente el OSO

```
const std::pair<const char*, const char*> DefaultGLTFModels[] =
{
    {"OSITO",          "models/Oso/glTF/OSITO.glTF"},
};
```

posteriormente se edito el código de la función UPDATE eliminando funciones innecesarias por el momento tales como la de cargar modelo manualmente.

```

void GLTFViewer::Update(double CurrTime, double ElapsedTime)
{
    if (m_CameraId == 0)
    {
        m_Camera.Update(m_InputController);
    }

    SampleBase::Update(CurrTime, ElapsedTime);
    UpdateUI();

    m_ElapsedTime = static_cast<float>(ElapsedTime);

    float YFov = PI_F / 4.0f;
    float ZNear = 0.1f;
    float ZFar = 100.f;

    float4x4 CameraView;
    if (m_CameraId == 0)
    {
        CameraView = m_Camera.GetRotation().ToMatrix() * float4x4::Translation(0

        m_RenderParams.ModelTransform = m_Camera.GetSecondaryRotation().To
    }
    else
    {
        const auto* pCameraNode = m_CameraNodes[m_CameraId - 1];
        const auto* pCamera = pCameraNode->pCamera;
        const auto& CameraGlobalTransform = m_Transforms[m_CurrentFrameNum

        // GLTF camera is defined such that the local +X axis is to the right,
        // the lens looks towards the local -Z axis, and the top of the camera
        // is aligned with the local +Y axis.
        // https://github.com/KhronosGroup/glTF/tree/master/specification/2.0#cam
        // We need to inverse the Z axis as our camera looks towards +Z.

```

```

float4x4 InvZAxis = float4x4::Identity();
InvZAxis._33      = -1;

CameraView = CameraGlobalTransform.Inverse() * InvZAxis;
YFov       = pCamera→Perspective.YFov;
ZNear      = pCamera→Perspective.ZNear;
ZFar       = pCamera→Perspective.ZFar;

m_RenderParams.ModelTransform = float4x4::Identity();
}

// Apply pretransform matrix that rotates the scene according the surface orien
CameraView *= GetSurfacePretransformMatrix(float3{3, 0, 1});

float4x4 CameraWorld = CameraView.Inverse();

// Get projection matrix adjusted to the current screen orientation
const auto CameraProj = GetAdjustedProjectionMatrix(YFov, ZNear, ZFar);
const auto CameraViewProj = CameraView * CameraProj;

float3 CameraWorldPos = float3::MakeVector(CameraWorld[3]);

auto& CurrCamAttribs = m_CameraAttribs[m_CurrentFrameNumber & 0x01];

const auto& SCDesc = m_pSwapChain→GetDesc();

CurrCamAttribs.f4ViewportSize = float4{static_cast<float>(SCDesc.Width), st
CurrCamAttribs.fHandness      = CameraView.Determinant() > 0 ? 1.f : -1.f;
CurrCamAttribs.mView          = CameraView;
CurrCamAttribs.mProj          = CameraProj;
CurrCamAttribs.mViewProj      = CameraViewProj;
CurrCamAttribs.mViewInv       = CameraView.Inverse();
CurrCamAttribs.mProjInv       = CameraProj.Inverse();
CurrCamAttribs.mViewProjInv   = CameraViewProj.Inverse();
CurrCamAttribs.f4Position     = float4(CameraWorldPos, 1);
CurrCamAttribs.fNearPlaneDepth = 0;

```

```

CurrCamAttribs.fFarPlaneDepth = 1;

if (m_bResetPrevCamera)
{
    m_CameraAttribs[(m_CurrentFrameNumber + 1) & 0x01] = CurrCamAttribs;
}

if (!m_Model→Animations.empty() && m_PlayAnimation)
{
    float& AnimationTimer = m_AnimationTimers[m_AnimationIndex];
    AnimationTimer += static_cast<float>(ElapsedTime);
    AnimationTimer = std::fmod(AnimationTimer, m_Model→Animations[m_Anim

    m_Model→ComputeTransforms(m_RenderParams.SceneIndex, m_Transform
    if (m_bResetPrevCamera)
    {
        m_Transforms[(m_CurrentFrameNumber + 1) & 0x01] = m_Transforms[m_C
    }
}

m_bResetPrevCamera = false;
}

} // namespace Diligent

```

Avance Investigacion AR

Con respecto al avance hecho para la el uso de gafas de realidad aumentada y el código de este ya se tuvo comunicación con una persona de Colivri que me brindara acceso a unas gafas, A pesar de esto estas estaban prestadas por lo que el primer día que se podían usar seria entre el 13 y 14 el de Mayo. Por dificultades de horario y de problemas en la instalación de programas necesarios para poder

realizar de manera satisfactoria una visita efectiva para el desarrollo del proyecto aun no se ha podido realizar primer acercamiento. Además se investigo acerca aplicación para Android compatible con el ejemplo Hello AR hecho en Diligent y se hizo intentos de instalarla para iniciar con la edición necesaria para iniciar el avance en realidad aumentada.

Por ultimo se están realizando pruebas para la instalación de `arcore` esto teniendo en cuenta el link puesto en la parte de referencias.

Evidencias Visuales

Las evidencias de videos se encuentran en el Readme del repositorio de Github, en este se muestra la animación original en Blender y como se ve esta en Diligent

Referencias

ARCore: <https://github.com/google-ar/arcore-android-sdk>